

C/C++

```

class vector_t {
public:
    vector_t(const int = 0);
    ~vector_t();
    void resize(const int);
    T get_val(const int) const;
    int get_size(void) const;
    void set_val(const int, const T);
    T& at(const int);
    T& operator[] (const int);
    const T& at(const int) const;
    const T& operator[] (const int) const;
    void write(ostream& = cout) const;
    void read(istream& = cin);
private:
    T *v_;
    int sz_;
    void build(void);
    void destroy(void);
};

```

C/C++

```

class matrix_t {
public:
    matrix_t(const int = 0, const int = 0);
    ~matrix_t();
    void resize(const int, const int);
    int get_m(void) const;
    int get_n(void) const;
    T& at(const int, const int);
    T& operator()(const int, const int);
    const T& at(const int, const int) const;
    const T& operator()(const int, const int) const;
    void multiply(const matrix_t<T>&,
                  const matrix_t<T>&);
    void write(ostream& = cout) const;
    void read(istream& = cin);
private:
    int m_, n_;
    vector_t<T> v_;
    int pos(const int, const int) const;
};

```

C/C++

```

class dll_t {
public:
    dll_t(void);
    ~dll_t(void);
    dll_node_t<T>* get_tail(void) const;
    dll_node_t<T>* get_head(void) const;
    int get_size(void) const;
    bool empty(void) const;
    void push_back(dll_node_t<T>*);
    void push_front(dll_node_t<T>*);
    dll_node_t<T>* pop_back(void);
    dll_node_t<T>* pop_front(void);
    dll_node_t<T>* erase(dll_node_t<T>*);
    ostream& write(ostream& = cout) const;
private:
    dll_node_t<T> *head_, *tail_;
    int sz_;
};

```

EJERCICIO 1 (RIERA, 2 puntos): Mochila 0-1 con Fuerza Bruta y Recursividad

Se desea resolver una instancia del problema de la **mochila 0-1** utilizando un algoritmo de **fuerza bruta recursivo**. Dada una lista de objetos, cada uno con un peso y un beneficio asociado, se pretende seleccionar un subconjunto de ellos de modo que el peso total no exceda un límite máximo preestablecido y el beneficio total sea máximo. Disponemos de:

- Un vector `weights[]` que contiene los pesos de 8 objetos.
- Un vector `prize[]` que contiene los beneficios de dichos objetos.
- Una capacidad máxima de la mochila `MaxWeight = 15`.

Se pide:

- Implemente una función recursiva que explore **todas las posibles combinaciones** de inclusión/exclusión de cada objeto (utilizando una variable binaria `x[i]` que toma el valor 1 si el objeto *i* se incluye en la mochila y 0 en caso contrario).
- La función debe guardar en un vector `best_x[]` la combinación factible que proporcione el **mayor beneficio total** sin exceder la capacidad máxima.
- Al finalizar, debe mostrar por pantalla:
 - El valor total del mejor beneficio encontrado.
 - El contenido de la mochila correspondiente a esa solución, representado por el vector `best_x[]`.

Datos:

```
C/C++
const int weights[] = {4, 6, 2, 8, 1, 3, 4, 3};
const int prize[]   = {4, 1, 4, 7, 3, 2, 5, 6};
const int MaxWeight = 15;
```

Cabecera de la función recursiva:

```
C/C++
void generate(vector<int> &x, int i, int n, int weight, int value);
```

Descripción de los parámetros:

- `x[]`: vector que almacena la combinación actual de inclusión/exclusión de objetos
- `i`: índice del objeto actual que se está considerando
- `n`: número total de objetos
- `weight`: peso total acumulado de los objetos incluidos hasta el momento
- `value`: beneficio total acumulado de los objetos incluidos hasta el momento

Programa principal:

```
C/C++
void generate_all(void)
{ vector<int> x(8);
  generate(x, 0, 8, 0, 0);
  cout << "Mejor valor: " << best_value << endl;
  cout << "Mejor mochila: ";
  for(int i{0}; i < 8; i++)
    cout << best_x[i] << " ";
  cout << endl;
}
```

Requisitos:

- No se permite el uso de bucles para generar combinaciones: la solución debe ser **completamente recursiva**.
- Se valorará el uso eficiente de parámetros y claridad en la estructura de la solución.

EJERCICIO 2 (RIERA, 2 puntos): Suma Recursiva de una Submatriz

Se desea implementar una función recursiva que calcule la suma de todos los elementos de una submatriz cuadrada de tamaño $n \times n$, comenzando en una determinada posición (i, j) de una matriz de enteros.

Se pide que implemente una función recursiva:

```
C/C++
int sum_submatrix(const matrix_t<int> &A, int i, int j, int n);
```

que devuelva la suma de todos los elementos de la submatriz de tamaño $n \times n$, con esquina superior izquierda en la posición (i, j) de una matriz A de tamaño fijo $N \times N$.

Suponga que:

- La matriz tiene tamaño $N = 5$
- Los índices estarán dentro de los límites, es decir, $i + n \leq N$ y $j + n \leq N$
- No se permite el uso de bucles: la función debe ser completamente recursiva

Ejemplo de llamada:

```
C/C++
5 5
1 2 3 4 5
6 7 8 9 10
11 12 13 14 15
16 17 18 19 20
21 22 23 24 25

// suma de la submatriz de 3x3 con esquina en (1,1)
cout << sum_submatrix(A, 1, 1, 3);

// Resultado esperado
Suma = 117
```

Cabecera de la función recursiva:

```
C/C++
int sum_row(const matrix_t<int> &A, int i, int j, int n);
int sum_submatrix(const matrix_t<int> &A, int i, int j, int n);
```

EJERCICIO 3 (RIERA, 1 punto): Suma iterativa de una Submatriz

Se desea resolver el mismo problema que en el ejercicio anterior, pero utilizando un enfoque iterativo.

EJERCICIO 4 (MARCOS, 1 punto): Concatenar una lista al revés con otra

Desarrollar el método de la clase `dll_t` denominado `rev_concat`, el cual concatena (une) todos los elementos en orden inverso de la lista pasada por parámetro sobre la lista invocante. La lista pasada por parámetro debe quedarse vacía después de la concatenación. La cabecera sería la siguiente:

```
C/C++
void dll_t<T>::rev_concat(dll_t<T>& L);
```

Por ejemplo, si tenemos las lista A: [1] ↔ [2] ↔ [3] y B: [4] ↔ [5] ↔ [6], la llamada `A.rev_concat(B)` daría como resultado A: [1] ↔ [2] ↔ [3] ↔ [6] ↔ [5] ↔ [4], y B vacía.

Nota: en la corrección de este ejercicio se tendrá muy en cuenta la eficiencia en el desarrollo de la solución propuesta.

EJERCICIO 5 (MARCOS, 1.5 puntos): Similitud de dos vectores `vector_t<double>`

Para calcular la similitud entre dos vectores A y B se usa la siguiente fórmula:

$$\text{Similitud}_{\cos}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Se pide desarrollar la función que devuelva la similitud entre dos objetos tipo `vector<double>` (uno invocante y el otro pasado por parámetro), y cuya cabecera es la siguiente:

```
C/C++
double vector_t<double>::cos_sim(const vector_t<double>& B);
```

EJERCICIO 6 (MARCOS, 2.5 puntos): Multiplicación de dos `dll_t<int>`

Dadas dos listas `dll_t<int>` que representan números enteros (un dígito por nodo), se pide desarrollar el método de multiplicación de ambas listas. Por ejemplo:

A: (head_) 2 ↔ 1 ↔ 4 ↔ 3 (tail_)

B: (head_) 5 ↔ 7 ↔ 8 (tail_)

1.238.654

El algoritmo procedería de la siguiente forma:

$$\begin{aligned} & (3 \times 10^0 + 4 \times 10^1 + 1 \times 10^2 + 2 \times 10^3) \times 8 \times 10^0 \\ + & (3 \times 10^0 + 4 \times 10^1 + 1 \times 10^2 + 2 \times 10^3) \times 7 \times 10^1 \\ + & (3 \times 10^0 + 4 \times 10^1 + 1 \times 10^2 + 2 \times 10^3) \times 5 \times 10^2 = 1.238.654 \end{aligned}$$

La cabecera del método será la siguiente:

```
C/C++
int dll_t<int>::multiply(const dll_t<int>& B);
```

Nota: en la corrección de este ejercicio se tendrá muy en cuenta la eficiencia en el desarrollo de la solución propuesta.

EJERCICIO 1

C/C++

```
void generate(vector<int> &x, int i, int n, int weight, int value)
{ if (i == n) {
    if (weight <= MaxWeight && value > best_value) {
        best_value = value;
        for(int j{0}; j < n; j++)
            best_x[j] = x[j];
    }
} else {
    x[i] = 0;
    generate(x, i + 1, n, weight, value);
    x[i] = 1;
    generate(x, i + 1, n, weight + weights[i], value + prize[i]);
}
}
```

EJERCICIO 2

C/C++

```
int sum_row(const matrix_t<int> &A, int i, int j, int n) {
    if (n == 0) return 0;
    return A.at(i,j) + sum_row(A, i, j + 1, n - 1);
}

int sum_submatrix(const matrix_t<int> &A, int i, int j, int n) {
    if (n == 0) return 0;
    return sum_row(A, i, j, n) + sum_submatrix(A, i + 1, j, n - 1);
}
```

EJERCICIO 3

C/C++

```
int sum_submatrix_iterative(const matrix_t<int> &A, int i, int j, int n)
{ int sum = 0;
  for (int row = 1; row <= n; ++row)
      for (int col = 1; col <= n; ++col)
          sum += A.at(i + row, j + col);
  return sum;
}
```

EJERCICIO 4

```
C/C++
template<class T>
void dll_t<T>::rev_concat(dll_t<T>& L)
{ while (!L.empty())
    push_back(L.pop_back());
}
```

EJERCICIO 5

```
C/C++
template<>
double vector_t<double>::cos_sim(const vector_t<double>& B)
{
    assert(get_size() == B.get_size());
    double s = 0, norm = 0, norm_B = 0;
    for (int i = 0; i < get_size(); ++i)
    { s += at(i) * B.at(i);
      norm += at(i) * at(i);
      norm_B += B.at(i) * B.at(i);
    }

    if (norm * norm_B == 0) return 0;
    return s / (sqrt(norm) * sqrt(norm_B));
}
```

EJERCICIO 6

```
C/C++
template<>
int dll_t<int>::multiply(const dll_t<int>& B)
{
    int value_A = 0, expo = 0;
    for (dll_node_t<int> *ptr = get_tail(); ptr != NULL; ptr = ptr->get_prev())
        value_A += ptr->get_data() * pow(10, expo++);

    expo = 0;
    int result = 0;
    for (dll_node_t<int> *ptr = B.get_tail(); ptr != NULL; ptr = ptr->get_prev())
        result += ptr->get_data() * pow(10, expo++) * value_A;

    return result;
}
```

Última modificación: martes, 10 de marzo de 2026, 11:06